Yggdrasil Platform — Bifrost Live Data Bridge Architecture

Mimir Labs Technical Publication

Document:	ML-WP-010	**Version:**	1.0
Author:	Christopher Gaither	**Date:**	March 2026
Classification:	Public		

0.1 Overview

Bifrost is the persistent synchronization layer of the Mimir Labs data platform. It operates as a continuous, event-driven service that maintains alignment between enterprise systems over time.

Where Ratatosk establishes semantic understanding and Ragnarok executes migration, Bifrost enforces ongoing coherence. It ensures that once systems are aligned, they remain aligned.

Bifrost is designed as a deterministic, system-agnostic bridge. It does not introduce new meaning. It propagates and enforces the meaning defined upstream.

0.2 Architectural Role

Bifrost completes the platform lifecycle:

- **Ratatosk:** discover and define meaning
- **Ragnarok:** migrate into aligned structure
- **Bifrost:** maintain alignment across systems

It converts a one-time migration outcome into a sustained operational state.

0.3 Design Principles

Bifrost is governed by four primary constraints.

Deterministic Behavior. All synchronization, routing, and conflict resolution logic is rule-based and predictable. No probabilistic or AI-driven behavior is introduced.

Event-Driven Architecture. Changes are propagated as events, not through polling or batch reconciliation. This enables near real-time system alignment.

Semantic Mediation. All routing decisions are based on a shared semantic model. Systems do not communicate directly; they communicate through a canonical interpretation layer.

Local-First Deployment. The system runs as a standalone service with no dependency on external orchestration or cloud infrastructure.

0.4 Synchronization Model

Bifrost supports multiple synchronization modes, allowing organizations to progress from evaluation to full production.

0.4.1 Mirror Mode

One-way propagation from source to target. The target is a read-only reflection of the source.

Used for validation and proof-of-concept. No risk is introduced to existing systems.

0.4.2 Unidirectional Mode

Source systems continue to feed a target system while allowing independent activity within the target.

Used when transitioning operational control to a new system without disrupting incumbents.

0.4.3 Bidirectional Mode

All connected systems participate in synchronization. Changes propagate across all systems according to defined ownership rules.

Used in full production environments where multiple systems remain active.

0.5 Core Architecture

0.5.1 Manifest-Driven Routing

Bifrost uses structured manifests as its routing foundation. Each entity and field is associated with a semantic definition that determines how data flows between systems.

Routing decisions are not hardcoded. They are derived from the semantic structure defined upstream.

0.5.2 Event Listeners

System-specific listeners detect changes within connected platforms. Each listener translates system-native events into a normalized internal representation.

Events are captured at field-level granularity to enable precise synchronization.

0.5.3 Sync Engine

The sync engine is responsible for distributing changes across systems. For each inbound event, the engine:

- determines applicable routes
- evaluates ownership rules
- propagates updates to all relevant systems

The engine maintains ordering, idempotency, and cycle prevention to ensure consistency.

0.5.4 Conflict Resolution

When multiple systems attempt to modify the same field, deterministic rules are applied.

Each field has a defined ownership model. Non-owner updates are evaluated against policy rules such as:

- timestamp precedence
- source priority
- manual intervention

Conflicts can be resolved automatically or escalated for human review.

0.5.5 Sync State Management

All synchronization activity is tracked in a local state store. This includes:

- last synchronization points
- pending changes
- conflict records
- system health metrics

State persistence enables recovery, replay, and auditability.

0.6 Execution Model

Synchronization follows a continuous event loop:

1. Detect change in source system
2. Normalize event
3. Evaluate routing and ownership
4. Apply conflict rules if necessary
5. Propagate change to other systems
6. Record outcome and update state

This model ensures that all systems converge toward a consistent representation of shared entities.

0.7 Observability and Control

Bifrost provides visibility into synchronization state at multiple levels.

System health includes listener status, queue depth, and error rates.

Entity-level status tracks whether records are in sync, pending, or diverging.

Periodic reconciliation processes verify consistency across systems and surface drift.

Structured logging enables traceability of events across the synchronization pipeline.

0.8 Security Model

Bifrost is designed for environments where data sensitivity is high.

- Credentials are encrypted at rest
- No plaintext secrets are stored
- All operations are auditable
- Deployment can occur entirely within controlled infrastructure

The system assumes that data movement between systems is a controlled and governed activity.

0.9 Platform Significance

Bifrost addresses a problem that traditional integration approaches do not solve.

Most integrations move data between systems. Bifrost maintains alignment between systems.

Without continuous synchronization, semantic alignment degrades over time as systems evolve independently.

Bifrost prevents that drift by enforcing consistency at the level of meaning, not just data movement.

Within the Mimir Labs platform, it represents the transition from migration to sustained coherence. It ensures that once an organization achieves alignment, it does not regress.